NORTHERN UNIVERSITY OF BUSINESS AND TECHNOLOGY, KHULNA
Department of Computer Science and Engineering

WEEK 01

Team Formation & Initial Project Idea

CSE 4104 · Software Development III · Section 7C · Fall 2026

SUBMISSION SUMMARY

Official Team Name	CSE4104-7C-T09
Section	7C
Project Title	UniBuddy: An AI Study Companion That Answers Questions From a Student's Own Course Material
Team Leader	Md Nafew Yeasin (11230321355)
Team Size	4 members
Submitted To	Assistant Professor Md. Riaz Mahmud
Submission Date	2 November 2026
GitHub Repository	github.com/hey-naf/cse4104-7c-t09-unibuddy

1 TEAM INFORMATION

The team consists of the members listed below. Roles follow the recommendation in the course handbook (Section 35.2); each member is responsible for one technical area and for being able to explain it during the final viva.

#	Full Name	Student ID	Role
1	Md Nafew Yeasin	11230321355	Team Leader
2	Mujibur Rahman Fahad	11230321568	AI Integration Lead
3	Abid Hasan	11230321553	Frontend Developer
4	M R Russell	11230321545	Backend Developer

2 PROJECT TITLE

"UniBuddy: An AI Study Companion That Answers Questions From a Student's Own Course Material"

In one line: *UniBuddy turns a student's own lecture PDFs into a tutor that answers questions about them and cites the page the answer came from, with the student's courses and deadlines tracked alongside.*

3 PRELIMINARY PROJECT DESCRIPTION

UniBuddy is a web application that turns a student's own lecture material into an interactive tutor. A student registers, adds the courses they are enrolled in this semester, and uploads the PDFs handed out in class: lecture slides, handouts, and reference notes. UniBuddy indexes those documents and lets the student ask questions about them in plain language. The answer is generated only from the uploaded material and is returned with the page number it came from, so the student can verify it against the original handout.

Alongside this, UniBuddy keeps a lightweight record of what each course requires: assignment deadlines, quiz and exam dates, and their status. The dashboard shows what is due this week, so the student arrives at the tutor already knowing which course needs attention.

The design deliberately separates the two halves. The organizer is ordinary software; it stores and sorts dates. The tutor is the part that cannot be built without AI, and it is the reason the application exists. A student holding four hundred pages of handouts with an exam on Thursday does not need another to-do list; they need to find the specific paragraph that answers the question they are stuck on, in the document their own teacher wrote.

The target users are undergraduate students at Northern University of Business and Technology, Khulna, who receive course material as PDFs and prepare for examinations largely from those documents. The interface is a responsive web application, usable on a phone between classes and on a laptop while studying.

4 WHY AI IS NEEDED IN THIS PROJECT

Three requirements of this system cannot be met by conventional software. They are the reason the project is not simply a task manager.

4.1 Searching by meaning rather than by keyword

A student's question rarely shares vocabulary with the passage that answers it. Asked why TCP needs a three-way handshake, a keyword search over a networking handout finds nothing useful, because the passage explains that both hosts must agree on initial sequence numbers before data is exchanged. The words do not overlap; the meaning does. Matching these requires semantic embeddings (a learned representation of meaning) rather than string matching. This is not an improvement to search; it is the only way search works on this material.

4.2 Answering from a closed, private document set

A general-purpose chatbot has never seen the student's handout. Asked about a lecturer's specific definition, it will produce a fluent, confident, and incorrect answer, because it is generating plausible text rather than retrieving the student's actual material. The problem is not model quality; it is the absence of the source. UniBuddy addresses this with retrieval-augmented generation: relevant passages are retrieved from the student's own uploads and supplied to the model as the only permitted basis for its answer. The

model's task narrows from knowing everything to reading these paragraphs and explaining them, a task it performs reliably.

4.3 Explaining at the right level

The same passage must be understandable to a first-year student and useful to a final-year student. Producing that explanation, adapted to the question actually asked, is a language task with no rule-based equivalent.

4.4 What this means for the design

The organizer module (courses, deadlines, calendar) requires none of this and could be built with plain CRUD operations. It exists to give the tutor context: knowing an exam is three days away changes which questions are worth asking. But it is not where the intelligence lives. If the AI were removed, UniBuddy would be a to-do list and would not be worth building. That is the test applied when deciding what this project is.

5 INITIAL TECHNOLOGY STACK

The stack below follows the course handbook's recommendations (Section 39) and is chosen so that one team can build and deploy it within the semester. These are initial selections and may be revised after the system design in Week 4.

Layer	Selection	Rationale
Frontend	React 18 with Vite, styled with Tailwind CSS	Component structure suits a dashboard; Vite keeps the build fast; Tailwind gives a responsive layout without a hand-written stylesheet.
Backend	Node.js with Express	Same language as the frontend, so the team does not context-switch; the handbook's recommended backend; direct access to AI provider SDKs.
Database	PostgreSQL, hosted on Supabase	Relational model fits users → courses → assignments → documents. The pgvector extension stores embeddings in the same database, avoiding a second service.
AI	Google Gemini API (chat and embeddings)	Multimodal with a usable free tier for development and a generous context window. OpenAI is kept as a fallback behind one provider interface.
Retrieval	Text chunking with pgvector similarity search	Grounds every answer in the student's own material and returns the page number needed for citation.
Authentication	JSON Web Tokens with bcrypt password hashing	Named explicitly in the handbook (Section 80.1); keeps sessions stateless and passwords unrecoverable.
File storage	Supabase Storage, private bucket	Uploaded handouts are personal. Per-user access policies prevent one student reading another's files.
Deployment	Vercel (frontend), Render (backend), Supabase (database)	Free tiers are sufficient for a semester project, and all three support secrets through environment variables.
Version control	Git with GitHub	Mandatory for the course. Branch strategy and commit conventions are documented in the repository README.

6 SIMILAR EXISTING SYSTEMS AND RESEARCH

Several existing products solve part of this problem. Each was reviewed to establish what already works well and where the gap lies.

System	What it does well	Limitation for our users
Notion / Notion AI	Strong organizer with built-in summarisation and a mature editor.	A generic workspace, not course-aware. Its AI does not ground answers in the student's uploaded PDFs with page citations, and setup is heavy for someone who only wants deadlines.
Google Classroom	The institution's own feed of assignments and materials.	Read-only for students. No personal task view, no study assistance, and no cross-course summary of what is due.
ChatGPT / Gemini (general chat)	Strong reasoning and explanation quality.	Has not read the student's course pack, so it answers confidently and without a source. No awareness of which courses the student takes.
Quizlet	Effective flashcards and practice tests.	Cards must be written by hand. No document upload and no question answering over lecture notes.
Anki	Spaced repetition that genuinely improves retention.	Manual card authoring and a steep setup curve. No AI assistance and no document ingestion.
Perplexity	Answers with citations and current web sources.	Cites the open web, not the student's closed course material. Cannot answer what a particular lecturer said about a topic.
Studocu / Course Hero	Large libraries of shared student notes.	Other people's notes, paywalled, and not guaranteed to match the syllabus being examined.

6.1 The gap this project addresses

No widely available tool combines three things: the student's own course documents, grounded answers carrying page-level references, and the deadline context of the current week. Each system above solves one part of that intersection. UniBuddy is deliberately scoped to the intersection itself, rather than to any one of the parts.

7 INITIAL SCOPE AND ROADMAP

7.1 In scope for the first working release

- Registration, login, and logout, with private per-user data.
- Student profile: university, department, and current semester.
- Course management: add, edit, and remove enrolled courses.
- Assignment, quiz, and examination tracking with deadlines and status.
- Dashboard showing what is due this week.
- Upload of lecture PDFs, organised by course.
- AI question answering grounded in the uploaded documents, with page references.
- Responsive interface for desktop, tablet, and mobile browsers.
- Persistent storage of all saved data in the database.

7.2 Deliberately deferred

The following remain part of the project's direction but are excluded from the first release, so that the core flow can be made reliable first:

- Quiz and flashcard generation from uploaded material.

- GPA and attendance tracking.
- A helpdesk answering questions over institutional documents such as handbooks and notices.
- Group project collaboration features.
- Bangla-language explanations.

Deferring these is a scope decision, not a claim that they are unimportant. They will be added only after the document-grounded tutor works end to end.

7.3 Planned milestones

Weeks	Milestone
1-2	Idea selection, research, proposal, and repository setup
3-4	SRS, system architecture, ER diagram, and API contract
5	Figma wireframes for the four core screens
6	Backend: authentication, database schema, and CRUD APIs
7	Frontend: React application connected to the live APIs
8	AI: document ingestion, chunking, embeddings, retrieval, and cited answers
9	Feature completion against the minimum requirements checklist
10	Testing: functional, API, interface, and AI-response quality
11	Deployment and environment-variable handling
12-14	Documentation, presentation, and final viva

8 DECLARATION

We confirm that this document describes our own original project idea. The systems listed in Section 6 were reviewed for comparison only; no code, content, or text has been copied from them. Every member of the team has read this submission and can explain the technical decisions it contains.

Name	Student ID	Signature
Md Nafew Yeasin	11230321355	
Mujibur Rahman Fahad	11230321568	
Abid Hasan	11230321553	
M R Russell	11230321545	